

Laboration 2 – Binära sökträd och rekursiva metoder

Uppgift 1 Generics

```
boxTest(new Box<Integer>);
```

kompilerar inte eftersom konstruktorn inte anropas korrekt. I Java måste ett objekt skapas med parenteser efter konstruktorn, alltså:

```
new Box<Integer>()
```

Jackson använder exemplet för att visa skillnaden mellan en typ och en faktisk objektinstans. `Box<Integer>` beskriver endast datatypen, men utan parenteserna skapas inget objekt i minnet. Därför uppstår ett syntaxfel vid kompilering.

Vid implementationen av ett binärt sökträd används följande generiska typ:

```
<T extends Comparable<? super T>>
```

Det betyder att typen `T` måste kunna jämföras med andra objekt. Detta är nödvändigt eftersom trädet behöver kunna avgöra om ett värde ska placeras till vänster eller höger i datastrukturen.

`extends Comparable` innebär att typen implementerar `compareTo`-metoden och därmed kan jämföras med andra objekt.

`? super T` innebär att jämförelsen även kan göras mot superklasser till `T`, vilket gör implementationen mer flexibel och generell.

PECS-regeln står för Producer Extends Consumer Super och används inom generics för att avgöra när `extends` respektive `super` ska användas.

I detta fall fungerar `Comparable` som en consumer eftersom metoden `compareTo` tar emot ett objekt att jämföra med. Därför används `super`.

Uppgift 2 Binärt sökträd

Min implementation av ett binärt sökträd består av klassen `BinarySearchTree` samt den inre klassen `BSTNode`.

Trädet är uppbyggt av noder där varje nod innehåller ett värde samt referenser till vänster och höger barn. Den första noden i trädet kallas roten.

Strukturen bygger på principen att mindre värden placeras till vänster och större värden placeras till höger. Detta gör det möjligt att effektivt söka efter, lägga till och ta bort element.

För att förstå hur ett binärt sökträd fungerar använde jag whiteboard under lektionerna. Det gjorde det lättare att visualisera hur noderna kopplas samman och hur rekursionen arbetar steg för steg genom trädet.

Hur add fungerar

När ett nytt element läggs till jämförs värdet med den aktuella nodens värde. Beroende på resultatet fortsätter metoden rekursivt till vänster eller höger delträd tills en tom plats hittas.

Pseudokod för add:

```
addRecursive(current, item)
```

om current är null

- skapa en ny nod med item

- öka size

- returnera den nya noden

jämför item med current.item

om item är mindre än current.item

- anropa addRecursive på vänster delträd

annars om item är större än current.item

- anropa addRecursive på höger delträd

annars

- ignorera dubbletten

returnera current

Basfall och rekursionssteg för add

Basfallet är när current är null. Då har metoden hittat en tom position i trädet där den nya noden kan placeras.

Rekursionssteget sker när metoden fortsätter nedåt i vänster eller höger delträd beroende på resultatet från compareTo.

Hur remove fungerar

Vid borttagning av ett element söker metoden först efter rätt nod. Därefter hanteras tre olika fall beroende på hur många barnnoden har.

Pseudokod för remove:

```
removeRecursive(current, item)
```

om current är null
returnera null

jämför item med current.item

om item är mindre än current.item
fortsätt rekursivt i vänster delträd

annars om item är större än current.item
fortsätt rekursivt i höger delträd

annars
om noden saknar barn
ta bort noden

annars om noden endast har ett barn
ersätt noden med barnet

annars
hitta det minsta värdet i höger delträd
ersätt current.item med detta värde
ta bort noden med det minsta värdet

returnera current

Basfall och rekursionssteg för remove

Basfallet är när current är null. Då betyder det att elementet inte finns i trädet.

Rekursionssteget sker när metoden fortsätter rekursivt nedåt i vänster eller höger delträd tills rätt nod hittas.

Dubbletter

Alla metoder använder compareTo för att navigera genom trädet. Det gör att add, search och remove arbetar enligt samma struktur och följer samma logik vid traversering av trädet.

När två värden är lika behandlas det som en dubblett och inget nytt element läggs till. På så sätt innehåller trädet endast unika värden.

Detta gör implementationen konsekvent och förenklar både sökning och borttagning eftersom varje värde endast kan förekomma en gång i trädet.

AI jämförelse

Jag använde AI som stöd under laborationen. AI hjälpte mig bland annat att generera delar av BST-koden, inklusive remove-metoden, samt att ta fram kvoter och resultat när jag fastnade.

Därefter anpassade jag koden så att den passade min egen implementation och programmeringsstil. Den AI-genererade lösningen var ibland mer avancerad än vad jag behövde, därför förenklade jag delar av koden och gick igenom den steg för steg för att förstå hur algoritmerna fungerar.

Uppgift 3 Tidskomplexitet

Hypotes

Jag tror att sökning går snabbare när talen är slumpade eftersom trädet då blir mer balanserat.

Jag tror att sorterade tal kommer ge sämre prestanda eftersom trädet blir obalanserat.

Resultat

1	n	Slumpade	Sorterade
2	5000	353800	18846200
3	10000	157600	38898300
4	20000	152200	failed
5	40000	143800	failed

Kvot

1	n	Kvot_Slumpade	Kvot_Sorterade
2	10000/5000	0.45	2.06
3	20000/10000	0.97	Failed
4	40000/20000	0.94	Failed

Analys

Resultaten för slumpade tal visar att tiden inte ökar särskilt mycket när trädet blir större. Det tyder på att trädet blir relativt balanserat och att sökningen är effektiv.

För sorterade tal ser man en tydlig skillnad. Tiden ökar mycket snabbare när antalet element ökar. Detta beror på att trädet blir obalanserat och börjar likna en länkad lista.

För större värden failed programmet. Detta beror på att trädet blev för djupt och rekursionen blev för stor.

Slutsats

Resultaten stämmer överens med hypotesen. Ett binärt sökträd fungerar effektivt när data matas in i slumpmässig ordning eftersom trädet då blir mer balanserat. Detta leder till snabbare sökning och bättre prestanda.

När data i stället matas in i sorterad ordning blir trädet obalanserat och börjar likna en länkad lista. Det gör att sökning, insättning och borttagning tar längre tid.

Resultaten visar därför att trädets struktur har stor påverkan på tidskomplexiteten och den övergripande prestandan.